

OpenSBI, bin, elf

最小可执行内核里, 我们需要完成以下两项关键工作:

1. 内核的内存布局和入口点设置
2. 通过 OpenSBI 封装输入输出函数

首先我们回顾计算机的组成:

计算机的基本组成包括CPU, 存储设备 (粗略地说, 包括断电后遗失的内存, 和断电后不遗失的硬盘), 输入输出设备, 总线。

在 QEMU 模拟的 RISC-V 系统中, QEMU会帮助我们模拟一块riscv64的CPU, 一块物理内存, 还会借助你的电脑的键盘和显示屏来模拟命令行的输入和输出。虽然QEMU不会真正模拟一堆线缆, 但是总线的通信功能也在QEMU内部实现了。

还差什么呢? 硬盘。

我们需要硬盘上的程序和数据。比如崭新的windows电脑里C盘已经被占据的二三十GB空间, 除去预装的应用软件, 还有一部分是windows操作系统的内核。在插上电源开机之后, 就需要将操作系统内核加载到内存中, 来运行操作系统的内核, 然后由操作系统来管理计算机。

问题在于, 操作系统作为一个程序, “把操作系统加载到内存里”这件事情, 不是操作系统自己能做到的, 就好像你不能拽着头发把自己拽离地面。

因此我们可以想象, 在操作系统执行之前, 必然有一个其他程序执行, 他作为“先锋队”, 完成“把操作系统加载到内存”这个工作, 然后他功成身退, 把CPU的控制权交给操作系统。

这个“其他程序”, 我们一般称之为bootloader. 很好理解: 他负责boot(开机), 还负责load(加载OS到内存里), 所以叫bootloader.

简单说明

对于有**复杂的读写时序要求**的设备，比如硬盘、U盘等，CPU是无法直接读取的，需要借助驱动程序来告知CPU如何与设备进行通信从而读到数据。

而这些驱动程序，就需要存储在一个不需要驱动程序，CPU就可以直接读取的地方，即一块像Memory一样，可以直接使用load/store命令可以访问，又不会担心数据会丢失的区域

在现代计算机中，有一些设备具备这样的特性，如早期的ROM芯片，以及后期相对普及的NOR Flash芯片

这些芯片与CPU连接之前，首先会进行初始化，在其中预置好对CPU的初始程序，即bootloader，然后再焊接至电路板上

在CPU启动时，会首先运行这些代码，用这些代码实现对硬盘、内存和其他复杂设备的读取

在QEMU模拟的riscv计算机里，我们使用QEMU自带的bootloader: OpenSBI固件，那么在Qemu 开始执行任何指令之前，首先要将作为 bootloader 的 OpenSBI.bin 加载到物理内存以物理地址 0x80000000 开头的区域上，然后将操作系统内核加载到QEMU模拟的硬盘中。

须知

在计算机中，**固件(firmware)** 是一种特定的计算机软件，它为设备的特定硬件提供低级控制，也可以进一步加载其他软件。固件可以为设备更复杂的软件（如操作系统）提供标准化的操作环境。对于不太复杂的设备，固件可以直接充当设备的完整操作系统，执行所有控制、监视和数据操作功能。在基于 x86 的计算机系统中, BIOS 或 UEFI 是固件；在基于 riscv 的计算机系统中，OpenSBI 是固件。OpenSBI运行在**M态 (M-mode)**，因为固件需要直接访问硬件。

RISCV有四种**特权级 (privilege level)**。

Level	Encoding	全称	简称
0	00	User/Application	U
1	01	Supervisor	S
2	10	Reserved(目前未使用，保留)	
3	11	Machine	M

粗略的分类：

U-mode是用户程序、应用程序的特权级，S-mode是操作系统内核的特权级，M-mode是固件的特权级。

那为何作为 bootloader 的 OpenSBI要被加载到 0x80000000 开头的区域上呢？加载到其他的地址空间可以吗？关于这个地址的选择，其实有软件工的工作，也有硬件人的工作。

实现细节

需要解释一下的是，QEMU模拟的这款riscv处理器的**复位地址**是0x1000，而不是0x80000000

所谓**复位地址**，指的是CPU在上电的时候，或者按下复位键的时候，PC被赋的初始值

这个值的选择会因为厂商的不同而发生变化，例如，80386的复位地址是0xFFFF0（因为复位时是16位模式，写成32位时也作0xFFFFFFFF0），MIPS的复位地址是0x00000000

RISCV的设计标准相对灵活，它允许芯片的实现者自主选择复位地址，因此不同的芯片会有一些差异。QEMU-4.1.1是选择了一种实现方式进行模拟

在QEMU模拟的这款riscv处理器中，将复位向量地址初始化为0x1000，再将PC初始化为该复位地址，因此处理器将从此处开始执行复位代码，复位代码主要是将计算机系统的各个组件（包括处理器、内存、设备等）置于初始状态，并且会启动Bootloader，在这里QEMU的复位代码指定加载Bootloader的位置为0x80000000，Bootloader将加载操作系统内核并启动操作系统的执行。

我们可以想象这样的过程：在Qemu启动时，OpenSBI作为bootloader会操作系统的二进制可执行文件从硬盘加载到内存中，然后OpenSBI会把CPU的"当前指令指针"(pc, program counter)跳转到内存里的一个位置，开始执行内存中那个位置的指令，也就是操作系统的指令。

那OpenSBI怎样知道要把操作系统加载到内存的什么位置呢？随便加载一个位置肯定是不可取的，因为加载之后OpenSBI还要把CPU的program counter跳转到操作系统所在的位置,开始操作系统的执行。如果加载操作系统到内存里的时候随便加载，那么OpenSBI怎么知道把program counter跳转到哪里去呢？难道操作系统的二进制可执行文件需要提供“program counter跳转到哪里”这样的信息吗？

也许你会觉得可以把操作系统的代码总是加载到固定的位置，比如总是加载到内存地址最高的地方。实际上，OpenSBI会将内核镜像 os.bin 加载到 Qemu 物理内存以地址 0x80200000 开头的区域上，并将CPU的控制权交给操作系统。为了正确地和上一阶段的 OpenSBI 对接，我们需要保证内核的第一条指令位于物理地址 0x80200000 处，因为这里的代码是地址相关的，这个地址是由处理器，即Qemu指定的。

一旦 CPU 开始执行内核的第一条指令，证明计算机的控制权已经被移交给我们的内核。

拓展 代码的地址相关性

这里我们需要解释一个小问题，为什么内核一定要被加载到0x80200000的位置开始呢？加载到别的位置行不行呢？

这要从高级语言被编译成为汇编和机器指令的过程说起了

以一行C语言代码为例，`int sum = 0;`，这句的意义是，在内存中分配一份可以容纳一个整数的空间，将其初始化为0。

那么，什么时候才会确定sum 被分配的空间的地址呢？在编译和链接的时候就会分配，假设这个地址是 `pa_sum`

接下来，当出现类似 `sum +=5;` 这样的语句时，相应的机器指令是，将 `pa_sum` 位置的值加载入寄存器，完成加法计算，再将这个值写回内存中对应的 `pa_sum` 处

而此时生成的指令，会将 `pa_sum` 的值（假设为0x55aa55aa），直接写入到指令的编码中，如 `load r1 0x55aa55aa`

如此一来，则这一段代码就成了**地址相关代码**，即指令中的访存信息在编译完成后即已成为绝对地址，那么在运行之前，自然需要将所需要的代码加载到指定的位置

与之相对的，自然会产生**地址无关代码**的技术需求，即在加载前才代码确定好被加载的位置，然后代码就可以在被加载处运行，这需要编译和加载时做一些处理工作，同学们可以思考一下这个技术需求的实现思路

在现有的ucore中，操作系统的加载实际上是由QEMU来完成，在 Qemu 开始执行任何指令之前，首先两个文件将被加载到 Qemu 的物理内存中：即作为 bootloader 的 OpenSBI.bin 被加载到物理内存以物理地址 0x80000000 开头的区域上，同时内核镜像 os.bin 被加载到以物理地址 0x80200000 开头的区域上。

在我们的Linux系统中，有两种不同的可执行文件格式：`elf` (e是executable的意思，l是linkable的意思，f是format的意思)和 `bin` (binary)。

`elf` 文件([wikipedia: elf](#))比较复杂，包含一个文件头(ELF header), 包含冗余的调试信息，指定程序每个section的内存布局，需要解析program header才能知道各段(section)的信息。如果我们已经有一个完整的操作系统来解析elf文件，那么elf文件可以直接执行。但是对于 OpenSBI来说，elf格式还是太复杂了。

`bin` 文件就比较简单了，简单地在文件头之后解释自己应该被加载到什么起始位置。

我们举一个例子解释elf和bin文件的区别：初始化为零的一个大数组，在elf文件里是bss数据段的一部分，只需要记住这个数组的起点和终点就可以了，等到加载到内存里的时候分配那一段内存。但是在bin文件里，那个数组有多大，有多少个字节的0，bin文件就要对应有多少个零。所以如果一个程序里声明了一个大全局数组（默认初始化为0），那么可能编译出来的

elf文件只有几KB, 而生成bin文件之后却有几MB, 这是很正常的。实际上, 可以认为bin文件会把elf文件指定的每段的内存布局都映射到一块线性的数据里, 这块线性的数据 (或者说程序) 加载到内存里就符合elf文件之前指定的布局。

那么我们的任务就明确了: 得到内存布局合适的elf文件, 然后把它转化成bin文件 (这一步通过objcopy实现), 然后加载到QEMU里运行 (作为bootloader的OpenSBI会干这个活)。下面我们来看如何设置elf文件的内存布局。